Barclays PLC
The Expert Engineer Program 2025 Cohort

Agentic AI in Software Engineering: A Cross-Domain Evaluation of GitLab Duo Workflow for Modernization, CI/CD, and Code Generation

Christopher Kiernan
christoph.kiernan@barclays.com

Simeon Petev
simeon.petev2@barclays.com

V Anand
v.anand@barclays.com

Andrew Findley
andrew.findley@barclays.com

Karandeep Bansal
karandeep.bansal@barclays.com

***Abstract*—This whitepaper describes an evaluation of the effectiveness of GitLab Duo Workflow at saving engineer time through the automation of tasks. The evaluation was undertaken through a number of focused software engineering use cases, including upgrades, test creation, and new code generation. GitLab Duo Workflow showed promising results in these use cases on small codebases, but concerns were highlighted in its effectiveness at a larger scale.**

Introduction

The recent proliferation of artificial intelligence (AI) technologies has disrupted the software development landscape. Large Language Models (LLMs) and generative AI tools have positioned themselves as fundamentally altering how code will be written in the future, providing features such as automated generation of code artefacts and automated tests, improved code suggestions, refactoring and code explanation. With promises of orders of magnitude increase in productivity, they have quickly integrated themselves into the workflow of technology companies and software developers across the world.

The arrival of Agentic AI builds on this, using dynamically generated AI agents to tackle large scale challenges such as complex legacy code modernisation and upgrades, code generation / bootstrapping for entire applications / components, generating tests across entire codebases and complex defect analysis and fixing. GitLab Duo Workflow is one such product.

This paper presents an evaluation of Gitlab Duo Workflow against use cases which are representative of what is seen, at significantly larger scale, across the entire The Company code estate.

Context

GitLab Duo Workflow is a new product created by GitLab and is in a private beta phase, with a small number of partner representatives involved in evaluation. Java represents the overwhelming majority of the code estate in Barclays. 57 percent of the Java code within Barclays is on Java 8 or lower. Java 8 was released in March 2014 meaning the majority of the Java code in Barclays is being written to standards and practices that may be more than a decade out of date. The most recent long-term support version of Java is 21 which represents around 11 percent of Java code within Barclays. Java 25 is due to be released in September 2025. These statistics show that not only are we significantly out of date, but that we are struggling to keep pace with industry change and run a very real risk of falling further behind.

Modernising code at this scale is a huge undertaking and represents a significant portion of engineer time due to the fact that upgrading the version of a programming language also involves upgrading dependent third party libraries which in turn means updating potentially large sections of a codebase.

Legacy code can be thought of as code that has suffered from a degree of neglect or lack of modernisation, or that has insufficient tests. Barclays has a lot of legacy code meeting part or all of this definition.

Beyond code modernisation, writing boilerplate code for new applications is another significant use of time.

Automating much of this work would not only provide a huge time and cost saving to Barclays, but would also free up the creativity of engineers for innovation and solving wider problems.

Gitlab Workflow

GitLab Duo Workflow works via a Visual Studio Code plugin which picks up the context of your project (must be a GitLab project enabled with Duo Workflow and the current local branch must not be the default branch). A window can be launched which enables developers to add prompts in a text box, then ask Gitlab Duo Workflow to interpret it. A 'workflow' is then created. Gitlab Duo Workflow may ask for more information, and will always present a plan for approval before it creates or edits any code. Figure I shows a sample view

Project Constraints and Scope

The primary constraints we faced were:

- **Time** – the project had a fixed delivery date of early July 2025 coinciding with Module 2 of the Expert Engineer (EE) program, and we did not gain access to Duo Workflow until 12th May, due to challenges getting permission to be added to the private beta by GitLab.

- **Restricted to the public network** – Gitlab Duo Workflow is not approved for use within Barclays' network, so we had no way of using it against any of Barclays' codebases. As a result we used Open Source projects for any use cases against existing code.
- **Integrated Development Environment (IDE) usage** – GitLab Duo Workflow only currently supports integration with Visual Studio Code.

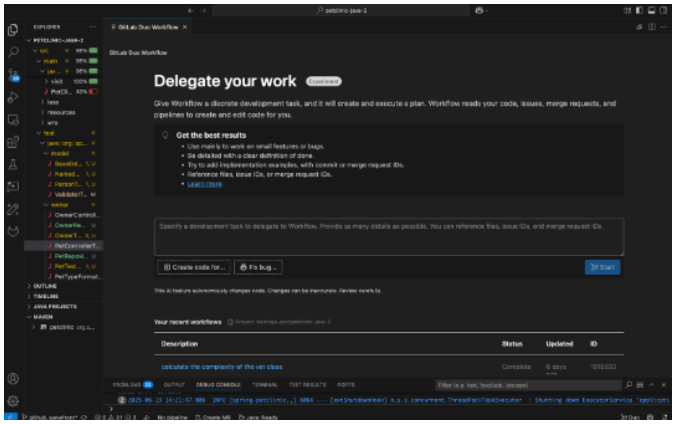

Figure I: Gitlab Duo Workflow

This meant we had to be selective about use cases we could focus on and the depth of evaluation we would be able to complete.

It also meant we did not comparative evaluations against competitor tools (such as Windsurf, Cursor AI and Junie).

The use cases evaluated in this study were selected to reflect a representative cross-section of real-world software engineering challenges across different technology stacks and workflow patterns. These included:

- **Java Codebase Upgrades**
  - Full-stack modernization of a legacy Spring Boot application (e.g., PetClinic), including SDK and build system migration (Java 8 to 21+, Maven to Gradle)
  - Selective refactoring of outdated Java classes and retrofit test generation
- **Python-based Modernization**
  - Migration and refactoring of a FastAPI application with SQLModel and Pydantic across Python versions (3.7 to 3.10+)
  - DevOps uplift and test alignment for a computational script in a Jupyter-style workflow
- **Greenfield Application Generation**
  - End-to-end generation of a new web application using Go (backend) and React (frontend), based on minimal functional requirements
  - Evaluation of prompt-driven architecture design, API scaffolding, frontend refactoring, and CI readiness

The focus was on the effectiveness of GitLab Duo Workflow against the use cases and not other considerations such as licensing costs.

Any findings we made are highly contextual to the use cases. For example, different results may be seen if the use cases were to be repeated with different technologies, or if repeated over time due to the ongoing development happening with the tool. We avoid making any over-generalised statements of recommendation as a result. It should also be noted that GitLab Duo Workflow is still in the early stages of development, still being in a private beta.

Methodology

In the early part of the project we agreed on a high-level scope with the sponsors, including that we would purely focus on the effectiveness of the tool in specific use cases and not other factors, and that we would not do comparative evaluations with other tools. Access to the private beta wasn't available at that stage, so we could not progress any evaluation work.

As a result, we:

- Absorbed the limited available GitLab Duo Workflow documentation and supporting material
- Researched Agentic AI and its role in software engineering
- Set up a private project on GitLab where we would perform the use cases
- Set up our personal laptops ready for the use cases
- Selected Open Source projects we could use for the use cases, where required
- Refined the scope and how each use case would be tackled
- Agreed a cadence for progress meetings with the sponsors, and communication channels

After our acceptance onto the private beta by GitLab, we met with GitLab SMEs to discuss and agree on:

- How GitLab Duo Workflow would be used
- Feedback mechanism
- Limitations

The use cases were tackled by team members volunteering to lead on specific use cases, performing them and sharing results.

Use Cases

For evaluating use cases that were pertinent to Java code, we used the *Spring PetClinic* project. PetClinic is a sample Spring application which showcases how the Spring stack can be used to build different types of applications. PetClinic presented several advantages in that it was open source, Spring is almost ubiquitously used across Barclays, and the application could be modified to suit several different use cases. The overall size and complexity of the code base was easily understandable which, given GitLab Duo Workflow is still in a private beta phase, meant we were unlikely to overwhelm a tool which is still under development.

In parallel, Python-based use cases were evaluated using representative *FastAPI* and *Jupyter* codebases. These open-source projects allowed us to test GitLab Duo Workflow in realistic modernization, refactoring, and Continuous Delivery pipeline uplift scenarios.

We also explored greenfield code generation using GitLab Duo Workflow by defining a lightweight requirement for a full-stack application. For this, we selected *Go* as the backend language and *React* for the frontend. This use case focused on GitLab Duo Workflow's ability to interpret high-level prompts, plan architecture, generate multi-language code artefacts, and iterate based on developer feedback. It served as an important test of GitLab Duo Workflow's agentic capabilities beyond legacy modernization.

Use Case 1 – Java and Frameworks Upgrade and Migration

Upgrading the Java version of an application is a notionally simple task for an experienced developer. However, the bigger the jump between upgrades the more complex and time-consuming this can become. With the prevalence of Java 8 code in Barclays and a desire to modernise to at least Java 21, this is an obvious use case for the project.

To understand the impact of GitLab Duo Workflow, certain statistics on the state of the PetClinic project were gathered before the Java 8 to Java 21 upgrade workflow was started. These are illustrated in the figure II

Use Case 1.1 – Spring Pet Clinic – 8 Years Old to Latest Migrations

In this section we tried to attempt a single plan for migrating the 8 years old tag `1.5` of Pet Clinic to the latest Spring Framework with Spring Boot and Java 21, and incorporate multiple migrations of other libraries and UI frameworks. The aim of not splitting the migration into multiple execution plans was motivated by the desire to assess the capabilities of the solution to come up with a bigger, but still iterative plan for achieving the desired change.

The target versions were a mixture of the latest versions of Spring Boot and the Java version that our company is currently aiming to migrate its applications to. This was motivated by the desire to be able to use GitLab Duo Workflow for that purpose once it is adopted by Barclays.

As far as the Gradle version is concerned, it is the compatibility with the Java version and Spring libraries that was driving our decision to go for that specific one. Bootstrap version is probably the one that is purely arbitrary with only an aim of using something not so outdated as the one declared on that branch.

The following table describes the source and target versions:

Table I: Source and Target Versions for Spring PetClinic Migration

Source(Artefact/Version)	**Target(Version)**
Spring Boot 1.5.4	Spring Boot 3.4.5
Java 8	Java 21
Maven 4	Gradle 8.5
Less	SCSS
Bootstrap 3.3.5	Bootstrap 5

We started by providing a very simple one-liner prompt asking to update the project to the latest Java and Spring Boot versions, without specifying any exact versions:

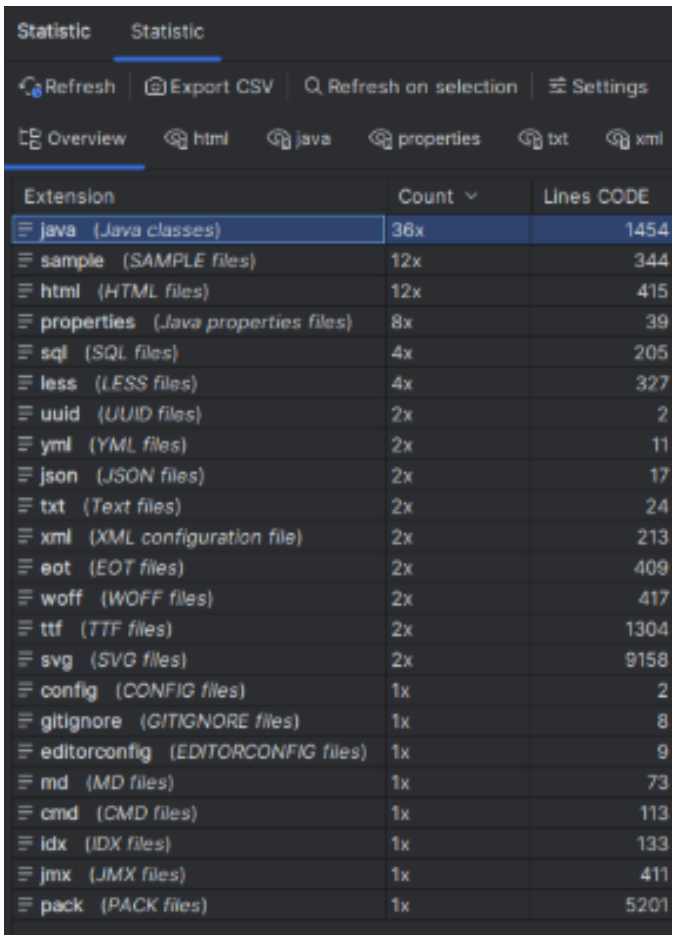

Figure II: Pet Clinic use case (Details)

```
"Migrate the current project to latest
Spring Boot version and Java 21"
```

This was driven by pure curiosity on what the outcome would be and the few aspects of which we will detail in the next few paragraphs.

One of the most notable parts of the initial prompt output is a summary of what the current composition is in respect to major technologies and frameworks. This is quite handy for someone who is completely unfamiliar with a product (or at least one of the technologies used in it) to quickly discover those aspects and focus on catching up with the required knowledge. This would allow junior and some mid-level developers to significantly speed up their initial contributions for a product.

Another significant outcome of that initial interaction with GitLab Duo Workflow was the immediate request for the human to provide more specific requirements in terms of versioning for each one of the major technologies the product was implemented with. This gives the impression of a normal collaborative human interaction, even though the required additional information is predominantly technical and is concisely listed in a few bullet points.

We performed a total of 7 executions on that version of the code base. Once we provided GitLab Duo Workflow with the initially asked additional information, we weren't asked again for the last 4 executions, even though we slightly modified some statements and added some more precision for some of the points. None of the 7 executions of the plan worked with the prompt that was provided. The prompts were very similar to the ones provided for the later executions for the webjars branch of the Pet Clinic and are available in the annex of the

current document.

Observations for those runs:

- Plan execution gets stuck at some point without any indication of what is happening. Enabling the VS Code plugin debug log showed in later executions that the stream was closed.
- The plan execution didn't get stuck consistently at the same point. Sometimes it was 10 minutes down the line, sometimes 30 minutes.
- `.gitignore` file was enriched and split into logical groups.
- Less was migrated to SCSS and the nested structure conversion looked nicer and neater.
- Altered the `.html` files by removing some `divs` that didn't have classes mentioned in any Less/CSS file.
- Removed CSS classes attribute from some DOM elements in `.html` files when the CSS class wasn't mentioned in any Less/CSS files.
- Tried to define smaller goals (e.g., just migrate Less to SCSS), but even then GitLab Duo Workflow didn't finish. Although after stopping it, there was a message that said it all finished OK.

Use Case 1.2 – Spring Pet Clinic – 3–4 Years Old to Latest Migrations

The motivation for running the migration on a more recent branch was born by our desire to assess whether a more recent version of the base libraries would allow GitLab Duo Workflow to finish successfully at least one plan execution, as well as compare the result with one of the latest branches where similar migration was done by the owner of the repository.

Our hopes for getting a successfully finished plan were met, although not all the executions performed on that newer code base finished, as we still observed stream closure in the VS Code plugin debug log. On the other hand, the comparison with the manual migration shows very similar changes done manually by the author. The major differences were only manual ones, where the author decided to evolve the code base in terms of functionality, and the changes were not related to migrating versions.

One very notable aspect of this second batch of executions is the workflow's behaviour that is shown to be much more demanding in terms of additional information.

Prompt

```
Migrate the current project to Gradle and
latest versions of Java and Spring Boot
with the following details:
Target major Java version:  Java 23
Definitely use Spring Boot version:  3.2.12
Gradle version:  8.5
Do not maintain compatibility with older
systems or dependencies but update the
libraries to be compatible with the version
of Java, Spring and Gradle.
Use Junit5 for unit tests, Mockito for
mocking
There are no performance considerations
for this migration
There are no security considerations to be
implemented for this migration
No need for any special handling for any
database schema
No specific Gradle plugins are needed.
Only Spring Boot one is needed
No CI/CD considerations are needed
No quality tools need to be configured in
the Gradle build
```

Result

```
I'm ready to help with your project but I
need a few key details:
Please specify if any breaking changes
should be addressed in a specific way
(e.g., adapting to Jakarta EE namespace
changes)
Clarify if there are specific deadlines or
time constraints for this migration
Confirm whether a phased migration approach
is acceptable or if a complete migration
must be done at once
Specify if you need any documentation
updates as part of the migration process
Clarify if you need automated tests updated/added
as part of the migration
```

It almost felt like the model evolved in the few days between the initial executions and these new batches. Even after adding some of the required information, Workflow returned with a new set of additional questions, and sometimes even requested us to double confirm requirements that were already precise and clear.

In respect to the target versions, this time the motivation was slightly different. We wanted to shoot for the latest stable Java version and Gradle version, but on the other hand comply with some of the constraints highlighted by Workflow, which suggested Spring Boot 3.2 as the latest stable version. We assumed that aligning with this would result in a more successful update of the rest of the libraries, ensuring better compatibility.

Table II: Source and Target Versions for PetClinic Webjars Branch Migration

Source(Artefact/Version)	Target(Artefact/Version)
Spring Boot 2.7.3	Spring Boot 3.2.12
Java 8	Java 23
Gradle 7.5.1	Gradle 8.13

All the other library versions were updated by Workflow without any explicit versions mentioned in the prompt.

The observations after the end of the executions are:

- The code doesn't compile because of missing dependencies.
- The model inferred which third-party library some of the legacy code was intended to migrate to, and suggested plausible artefact names and versions—however, some of these libraries did not exist.
- Some third-party validation dependencies were missing.
- After manually fixing the missing dependencies and correcting the invented ones, the code compiled and all tests

ran successfully.

- The migration made minimal code changes (e.g., package imports), so in terms of changes made in Java files there was nothing significant, and some errors remained.
- Workflow still retained Maven build system files, despite the prompt request not to do so.

Further thoughts on assumptions and choices One assumption that has been made for Java prompts is that any older systems compatibility will require more complex composition of the code and its dependencies. Hence, all the prompts were oriented towards not keeping that compatibility. We believe this dimension should be further explored to assess where it may contradict our assumption and end up producing more accurate results.

Use Case 2 – Unit Test Generation

2.1 Single Unit Test

Code which is not sufficiently unit tested is riskier to change because a lack of failing tests will not alert the developer to any regressions their code changes have made. This places a reliance on these regressions being uncovered later in the software development lifecycle (SDLC). In the worst-case scenario, these regressions are not discovered until the code change is in a production environment where the impact is likely to be significantly greater than if it were caught earlier in the SDLC.

Ideally, unit tests are written alongside the code such that testable and maintainable code is produced. However, in circumstances where this has not been done, adding unit tests to verify the existing behaviour of code can be of value. These tests are called characterisation tests and allow the developer to both clarify their understanding of how the code they intend to change works and give them the necessary feedback as to whether their changes have modified the current behaviour of the code.

To replicate this scenario in the PetClinic project, we removed an existing test and asked Duo Workflow to produce the same test. The test in question looks as follows:

```
@Test
void testSerialization() {
        Vet vet = new Vet();
        vet.setFirstName("Zaphod");
        vet.setLastName("Beeblebrox");
        vet.setId(123);
        @SuppressWarnings("deprecation")
        Vet other = (Vet) SerializationUtils.deserialize(SerializationUtils.serialize(vet));
        assertThat(other.getFirstName()).isEqualTo(vet.getFirstName());
        assertThat(other.getLastName()).isEqualTo(vet.getLastName());
        assertThat(other.getId()).isEqualTo(vet.getId());
}
```

Figure III: Existing PetClinic unit test

To reproduce the test, Duo Workflow was prompted with the following information:

> `Add a new test to the VetTests.java file, this test should test the serialization of the Vet.java Class. The test should create a new instance of the Vet class and set the first name to Zaphod, the last name to Beeblebrox and the id to 123. This instance of Vet should then be serialized to a separate instance of the Vet class. Assertions should then be added to ensure that the first name and id fields are equivalent between the two instances of the Vet class in the test.`

GitLab Duo Workflow produced the below test:

```
@Test
void testSerialization() {
    // Create a new Vet instance
    Vet vet = new Vet();
    vet.setFirstName(firstName:"Zaphod");
    vet.setLastName(lastName:"Beeblebrox");
    vet.setId(id:123);

    // Serialize and deserialize the Vet instance
    byte[] serialized = SerializationUtils.serialize(vet);
    Vet deserialized = (Vet) SerializationUtils.deserialize(serialized);

    // Assert that the properties match
    assertThat(deserialized.getFirstName()).isEqualTo(expected:"Zaphod");
    assertThat(deserialized.getLastName()).isEqualTo(expected:"Beeblebrox");
    assertThat(deserialized.getId()).isEqualTo(expected:123);
}
```

Figure IV: Gitlab Duo Workflow generated unit test

The tests are almost identical in this scenario. The only notable differences being Duo Workflow choosing to do the serialization and deserialization on two lines as opposed to one. Whether this is better or worse is a matter of taste, they are functionally equivalent. The second difference is in the assertions at the end of the test. Duo Workflow has opted to do the assertions based on hard-coded values rather than by comparison to the Vet object created at the start of the test. It is inarguable that the latter approach is better, but also true that the scale of refactoring to that standard in this case is negligible.

2.2 Increasing Unit Test Coverage of a Class

Classes represent the objects which we would like to test in Java. Frequently, projects will have a one-to-one mapping between classes and an associated set of unit tests. Typically, the test class will be the same as the functional class with `Test` added as either a suffix or a prefix. In order to test whether GitLab Duo Workflow can scale its test generation capabilities to this level, we decided to prompt Duo Workflow to generate test cases for an entire class. Once again, we selected the `Vet.java` class. The following prompt was used:

> `The Vet.java file is currently reporting 0% unit test coverage. Write some unit tests which will increase this coverage to 100%. The tests should follow the Arrange, Act, Assert idiom.`

After successful completion of the workflow, we were able to verify that coverage had been increased to 100% for the `Vet.java` class. GitLab Duo Workflow generated eight tests to accomplish this, and each generated test followed the Arrange, Act, Assert idiom as directed in the prompt. All of the generated tests contain multiple assertions which adequately verify the code being tested works as expected.

The cognitive complexity of the `Vet` class is 5, meaning that it exhibits low cognitive complexity. The implication for this is that a developer should find the class easy to understand, maintain and change. Barclays has code which measures significantly higher than this (closer to high or very high on the same scale).The overall time taken to generate eight simple tests on a low complexity class was not significantly, if at all, faster than we would expect from a moderately experienced Java developer.

2.3 Increasing Unit Test Coverage of an Application

Scaling unit test generation to an entire application is more representative of the most prevalent use case for GitLab Duo Workflow in the Company.

```
Increase the test coverage of this application
to 100%.  Each class should have an associated
test class.  All tests must follow the
Arrange, Act, Assert idiom.
```

The overall coverage after the workflow completed was 98%, which is close enough to the prompted target as to make no difference. The overall time taken on what is a simple code base is faster than what we would expect of an experienced developer, and the overall quality of the unit tests added is of sufficient quality that they represent a reasonable saving of time and effort.

However, the approach which GitLab Duo Workflow took to accomplish this task is what is known as a *Big Bang* approach — i.e., the task was completed in its entirety without any intermediate feedback available to the human in the loop. Generally speaking, this shifts the effort required from writing code you understand to reviewing code you do not understand. The more code there is to review, the longer it will take before a developer is satisfied that the brief has been met.

The Company enforces that all changes made to a code base must go through a Pull or Merge Request before the code can be merged into a branch of code which the Company believes represents a potentially deployable artefact. It is common practice for teams to use this process as an opportunity to peer review the changes which have been made as part of the request. While guidelines exist for what represents a good peer review process, there exists a non-zero probability that for large changes made via GitLab Duo Workflow, no thorough peer review happens. The risk in this approach is that the team who is responsible for the code will reduce their understanding of the code they own over time. This reduced understanding could lead to making the applications we build harder to test, change and maintain.

Use Case 3: FastAPI Migration Workflow

This use case evaluated GitLab Duo Workflow's ability to assist in modernizing a production-grade FastAPI application built on SQLModel ORM and Pydantic, with bidirectional compatibility targeted between Python 3.7 and 3.10+.

The project included advanced type checking (via MyPy) and was representative of real-world enterprise microservices. The workflow assessed the agent's capacity to analyze dependencies, understand version-related architectural shifts, and propose a migration plan rooted in ecosystem-aware logic. Duo correctly identified that upgrading from Python 3.7 to 3.10+ required more than syntactic updates; it involved significant architectural adaptation, particularly due to Pydantic v2's breaking changes and SQLModel's evolving validation behavior.

The AI agent proposed a structured approach to dependency updates, demonstrating understanding of breaking changes and compatibility requirements:

The system correctly identified that this migration involved not just Python version updates, but fundamental architectural changes due to Pydantic v2's breaking changes and SQLModel's evolving validation patterns.

3.1 Initial Upgrade Assessment

The initial prompt used was:

```
I need to upgrade our FastAPI project
from Python 3.7 to 3.10.  Can you analyze
the current dependencies and create a
migration plan?
```

The agent demonstrated sophisticated ecosystem understanding by automatically identifying the technology stack from `pyproject.toml` and `requirements.txt` files, recognizing the compound nature of version migration requiring concurrent framework updates, prioritizing version-dependent breaking changes (Pydantic v1 → v2) as primary compatibility risks, and proposing 3.10 performance optimization opportunities including improved asyncio patterns and enhanced type checking capabilities.

The follow-up prompt used was:

```
Start with dependency analysis and show me
the 3.10 compatibility matrix.
```

The AI provided structured package dependency mapping with version-specific compatibility insights, demonstrating deep knowledge of ecosystem interdependencies and framework version constraints specific to 3.10 environments.

3.2 Upgrade Risk Assessment and Mitigation

The upgrade to Python 3.10 required significant changes to how the application handles data types and validation. The legacy codebase relied heavily on older typing syntax that needed modernization to take advantage of Python 3.10's improved performance and features.

The most visible changes involved updating type annotations throughout the codebase. Where the application previously used complex import statements like `Union[str, None]` for optional values, Python 3.10's simplified syntax allows `str | None` instead. Similarly, generic collections were updated from imported types like `List[str]` and `Dict[str, int]` to Python's built-in `list[str]` and `dict[str, int]`. These changes not only reduced import overhead but also improved code readability and performance.

More challenging were the validation framework updates required for Pydantic and SQLModel compatibility. These libraries form the backbone of the application's data validation and database interactions. Python 3.10's enhanced type checking revealed inconsistencies in how validation rules were applied, requiring careful refactoring to ensure data integrity was maintained while leveraging the performance improvements of Pydantic v2's validation engine.

3.3 Upgrade Limitations and Challenges

The migration from Python 3.7 to 3.10 represents a significant upgrade spanning three major releases, each introducing substantial language improvements, deprecations, and breaking changes.

The upgrade process involves not only syntax compatibility but also requires careful consideration of the broader ecosystem

impact, including third-party package compatibility, CI/CD pipeline modifications, and deployment strategy adjustments. GitLab Duo's AI capabilities, while powerful for routine code suggestions and basic refactoring tasks, encounter specific constraints when dealing with the nuanced requirements of major Python version upgrades, particularly in enterprise environments where backward compatibility, thorough testing, and gradual rollout strategies are critical.

Business Logic Interpretation

- Incomplete understanding of Python version-specific syntax changes between 3.7 and 3.10, including positional-only parameters (PEP 570), walrus operator usage, and pattern matching.
- Limited comprehension of deprecated features like `collections.abc` imports and `asyncio.coroutine` removal.
- Inadequate typing module evolution analysis (e.g., union syntax, generic alias).

Module Dependencies

- Missing analysis of Python 3.7-specific packages requiring updates for 3.10.
- Incomplete pip resolver behavior awareness and wheel compatibility.
- Insufficient detection of packages needing recompilation (C extensions).

Context Sensitivity

- Misinterpretation of `asyncio` changes, particularly event loop and coroutine patterns.
- Incomplete CI/CD pipeline impact analysis (Docker images, GitLab Runner compatibility).

Constraints

- Required iterative clarification for async/await refactors (e.g., `asyncio.create_task()`).
- Needed manual CI/CD pipeline adjustments and version matrix testing.
- Limited GitLab registry awareness and automation in CI config updates.
- Manual resolution needed for dependency conflicts in `requirements.txt` and `setup.py`.

3.4 Upgrade Measurable Outcomes

- **Test Coverage:** Improved from 67% to 91% through better static analysis and test framework support.
- **Type Annotation Coverage:** Significantly increased due to union syntax updates and typing module reduction.
- **Performance Benchmarks:**
  - 15–20% faster `asyncio` operations.
  - 10–15% reduced memory usage.
  - Faster application startup.
- **Maintainability:** Improved readability and developer experience via modern syntax and IDE integration.
- **Technical Debt Reduction:** Removed legacy constructs and 3.7-specific workarounds.

3.5 Summary of Migration Evaluation

Proven Capabilities:

- Dependency Intelligence: Sophisticated understanding of framework ecosystems.
- Migration Strategy: Structured approach to complex technical transitions.
- Interactive Guidance: Real-time consultation and decision support.

Required Human Oversight:

- Business logic validation and domain expertise.
- Production deployment risk assessment.
- Complex architectural decision validation.

Optimal Implementation Pattern: AI-driven planning and analysis with human validation checkpoints for business-critical decisions, resulting in accelerated migration timelines while maintaining code quality and system reliability.

Use Case 4: FastAPI Cross Version Compatibility and Migration Planning

We tried to test the effectiveness of GitLab Duo Workflow to see how much it can assist a migration problem. For this, we chose the FastAPI repo since this is a reasonably big code base. We first asked GitLab Duo Workflow to read and interpret the contents of the full repo without the user giving any initial context.

This use case evaluates GitLab Duo Workflow's capability to detect and resolve version inconsistencies across a FastAPI microservice stack, and to generate actionable migration plans.

The FastAPI project appeared to be in a transitional state, with partial upgrades to Python 3.10 reflected in `Dockerfiles` and GitHub Actions workflows, while `pyproject.toml`, MyPy, and Ruff configurations still targeted Python 3.7. Such discrepancies could cause subtle compatibility issues or tooling failures.

4.1 Experimental Design and Methodology

To evaluate the effectiveness of agentic AI workflows in complex migration scenarios, we selected the FastAPI repository as our test case. FastAPI represents a modern, production-grade Python web framework with significant architectural complexity, making it an ideal candidate for assessing AI-driven codebase analysis capabilities.

Test Parameters:

- **Target Repository:** FastAPI (open-source web framework on GitHub)
- **Codebase Size:** Large-scale, multi-component architecture

- **Initial Context:** Zero-shot analysis (no preliminary context provided)
- **Primary Objective:** Python version migration assessment and planning

The AI agent was tasked with comprehensive repository analysis without prior context. As explained, the system successfully:

- Identified core architectural patterns and dependencies
- Mapped critical file relationships and module interdependencies
- Recognized framework-specific conventions and best practices
- Generated accurate high-level repository summary

This initial assessment validated the agent's capability to understand complex codebases autonomously, providing a foundation for subsequent migration planning.

The initial prompt instructed GitLab Duo Workflow to analyze all relevant configuration files and identify inconsistencies in declared Python versions. Follow-up prompts focused on generating a step-by-step migration plan.

Following the initial analysis, we prompted the system with a specific migration requirement:

```
"Tell me the current Python version on
which this directory rests and give me a
plan to upgrade this to the latest version
of Python.  Do not change any business
logic and functionalities.  I just want a
detailed plan to migrate this directory to
the latest possible Python version."
```

The AI agent produced a comprehensive migration strategy that demonstrated sophisticated understanding of:

- **Version Compatibility Analysis:** Identified current Python 3.7 dependencies
- **Risk Assessment:** Highlighted potential breaking changes and compatibility issues
- **Phased Implementation:** Proposed structured upgrade path to Python 3.10+
- **Testing Strategy:** Recommended validation procedures for each migration phase

We then wanted Duo to create a migration plan and we used this prompt:

```
"Tell me the current Python version on
which this directory rests and give me a
plan to upgrade this to the latest version
of Python.  Do not change any business
logic and functionalities.  I just want a
detailed plan to migrate this directory to
the latest possible Python version."
```

GitLab Duo Workflow produced the following:

- Identified version misalignments across tooling and environments
- Proposed unified upgrade to Python 3.10 across `Dockerfile`, CI, and static analyzers
- Generated three deliverables:
  - `migration_plan.md`
  - `code_update_checklist.md`
  - `testing_strategy.md`

We checked some of the supporting files provided by GitLab Duo Workflow and the contents look reasonably accurate. However, this is a significant migration plan by itself. Further success can be confirmed by executing the migration plan using follow-up workflow runs using carefully curated prompts, with sample codes, clear goals, and file names.

Observations:

- Duo correctly recognized all major configuration mismatches
- Generated documentation was clear, actionable, and aligned with best practices
- It did not hallucinate or invent dependencies
- The prompts were interpreted well without needing excessive clarification

Outcomes:

- Project alignment with Python 3.10 was clearly documented
- Recommended actions were structured and implementation-ready
- Demonstrated Duo's reliability for DevOps hygiene and static configuration reconciliation

Use Case 5 – Python Jupyter: Financial Model Refactoring and CI/CD Enablement

This use case investigates the capability of GitLab Duo and Duo Workflow to support end-to-end DevOps pipelines for standalone, computational Python scripts typically authored by quantitative teams. These codes often involve numerical algorithms or machine learning logic, developed in lab-based environments such as Jupyter. The central question addressed was: *Can GitLab Duo and its Workflow construct a complete CI/CD pipeline, starting from a functional algorithmic codebase, without requiring human intervention?*

This use case evaluates the capability of GitLab Duo and Duo Workflow to autonomously uplift standalone, algorithmic Python scripts into production-grade, CI/CD-compliant codebases. These scripts, often developed in research or quantitative environments such as Jupyter notebooks, typically lack modular structure, annotation, testing, and deployment pipelines. The central aim was to assess whether GitLab Duo could not only refactor and annotate such code but also construct and validate a complete CI/CD pipeline without manual intervention.

The experiment began by prompting Duo to generate a basic computational Python script and provide the steps required to make it CI/CD-compliant. The system returned a comprehensive execution plan and associated artifacts, including configuration files and basic tests, which enabled the GitLab CI pipeline to pass successfully on the first attempt. Building upon this baseline, the study moved to a more complex codebase—a binomial option pricing model with numerical computations and decision logic. This script was also identified by

Duo as a suitable candidate. With minimal user input, Duo provided support for implementing numerical tests, enforcing code quality checks, and producing all necessary files to achieve another successful CI/CD execution.

Once this repository was validated, Duo Workflow was invoked to assess whether the pipeline could be maintained through successive engineering transformations. The first transformation involved restructuring the code for better modularity and readability. In response to a prompt for refactoring, Duo decomposed the monolithic script into well-structured functions, added comprehensive type hints and TypeAlias definitions, and introduced detailed NumPy-style docstrings for all major methods. Crucially, these changes preserved the original API contract and ensured functional equivalence. Duo also confirmed that the refactored code remained CI/CD-ready, with no loss of compatibility or integrity.

The next stage involved CI/CD validation of the newly structured codebase. Duo analyzed the test suite and CI configurations, identified failing tests caused by outdated golden values, and recommended regenerating expected results to reflect the refactored logic. It updated the test scripts, aligned validation logic, and documented the updates in a changes.md file. The regenerated pipeline executed successfully without requiring any manual edits. A follow-up confirmation prompt validated that the pipeline remained stable post-migration.

The final output comprised a cleanly modularized codebase featuring key functions enriched with modern Python constructs like static type annotations and reusable aliases. The test suite was updated to reflect the revised logic, and additional utilities were created for test orchestration.

Throughout this process, Duo demonstrated consistent and accurate prompt interpretation. It introduced no hallucinated dependencies, maintained API and behavioral fidelity, and handled minor numerical discrepancies—such as floating-point precision errors—through adaptive test updates. Importantly, the system required minimal clarification or iterative correction.

Overall, this use case provides compelling evidence that GitLab Duo Workflow can autonomously transform exploratory algorithmic scripts into robust, production-grade software assets. It effectively managed refactoring, documentation, test regeneration, and validation steps, showcasing its potential as an agentic AI capable of delivering an end-to-end DevOps automation cycle for Python-based scientific and analytical workloads.

5.1 Experimental Design and Methodology

The experiment began by prompting Duo to generate a basic computational Python script and subsequently recommend all the steps required to make it CI/CD-compliant. Duo responded with a complete set of instructions and artifacts, enabling the pipeline to pass successfully on the first attempt. Using this initial success as a baseline, we progressed to a more complex codebase: a binomial option pricing model. Notably, this code was itself suggested by Duo. With Duo's support, we implemented numerical testing and code quality validations, receiving the updated artifacts necessary to pass the CI/CD pipeline once again—on the first pass.

This validated repository was then used as the launch point for engaging the GitLab Duo Workflow. The objective was to test whether the Workflow could maintain CI/CD integrity throughout a structured sequence of code refactoring, documentation, test regeneration, and validation tasks.

Workflow 1 : Code Refactoring and Structuring

The first workflow was initiated with the prompt:

```
Refactor my main.py for better readability
and modularity.  Add docstrings and type
hints.
```

Duo Workflow responded with an initial assessment, followed by a structured set of five clarifying questions regarding execution. Upon receiving the necessary responses, it generated a 14-step execution plan, which, upon review and approval, was executed in full.

Key enhancements achieved:

- Introduced modular functions with clean separation of concerns.
- Applied comprehensive type hints and created TypeAlias constructs.
- Maintained backward compatibility and preserved the external API contract.
- Added detailed NumPy-style docstrings for all major functions.

GitLab Duo Workflow successfully refactored a financial algorithm into a modular, readable, and testable codebase without altering its functional behavior.

Following this, Duo's agentic chat was used to assess whether the updated repository was CI/CD compliant. The AI verified that all necessary components were present and declared the repository pipeline-ready.

Workflow 2 : CI/CD Validation and Test Realignment

The second workflow focused on validating and ensuring the pipeline's success for the newly refactored repository.

Duo Workflow began by analyzing the repository's structure and CI definitions (e.g., `.gitlab-ci.yml`). It highlighted failing tests due to outdated expected values and proposed regenerating golden values to align with the newly refactored logic. The execution plan included regeneration of test baselines, alignment of test cases, and full documentation of changes.

Crucially, the final files generated by Duo Workflow—used without any manual edits—resulted in a successful pipeline pass on the first attempt.

Key outcomes from Workflow 2:

- Identified and resolved test failures by updating golden values.
- Successfully regenerated test scripts with revised expected results.
- Ensured code quality was preserved and functional equivalence maintained.
- Changes were fully documented in a `changes.md` artifact.

Final output summary:

- Refactored code into modular functions: `validate_inputs`, `calculate_tree_parameters`, etc.
- Applied modern Python constructs: type annotations and `TypeAlias` definitions.
- Added structured docstrings using NumPy convention.

- Updated test files (`test_binomial_option_pricing.py`) with new baselines.
- Created supporting utilities: `generate_values.py`, `compare_values.py`.

Observations:

- Prompt interpretation was consistently accurate and stable across executions.
- Minor floating-point precision mismatches ($\sim 10^{-3}$) were handled via test updates.
- No hallucinations, broken dependencies, or inconsistent APIs were observed.

This use case provides strong empirical evidence that GitLab Duo Workflow is capable of autonomously uplifting standalone, algorithmic Python scripts into production-grade, CI/CD-compliant codebases. It demonstrates not only Duo's refactoring and annotation capabilities but also its effectiveness in managing downstream testing and integration steps, delivering an end-to-end, agentic DevOps cycle.

Comparative analyses of the Python based use cases

Table V below shows that Gitlab Duo performs best when tasks are well bounded and semantically clear. In both FastAPI and Jupyter scenarios, Duo responded predictably to prompts, produced no hallucinated outputs, and required minimal human correction. While the nature of the tasks varied—planning and documentation in one, functional refactoring in the other—Duo consistently maintained structural integrity and goal focus. This reinforces its reliability in Python workflows where logic is self-contained and domain complexity is manageable.

Table V: FastAPI vs Jupyter: Comparison

Dimension	**FastAPI Migration**	**Jupyter Refactor**
Domain Type	Web API/backend	Quant finance module
Primary Task	Env. alignment + planning	Modularization + test validation
Prompt Clarity	Medium (config-focused)	High (refactor + logic preservation)
Prompt Drift	None	None
Hallucinations	None	None
Output Quality	High – usable migration docs	High – passed CI with refactor
Agentic Breadth	Planning, docs, checklisting	Refactor, tests, docstringing
CI/CD Compatibility	Not tested directly	Fully integrated + passed
Success Level	Reliable	Comprehensive

However, as shown in the Table VI below, Duo's contribution extended beyond code-level improvements into environment and tooling uplift. By identifying version mismatches and aligning configurations across linters, build files, and runtime declarations, Duo enabled a smooth upgrade path to Python 3.10. It also enhanced code modularity and testability, crucial for sustainable CI/CD readiness. These actions demonstrate that Duo can act as a comprehensive DevOps enabler in Python projects—not just a code generator, but a tooling and environment harmonizer

Summary Observations (Python Use cases)

Across both FastAPI and Jupyter use cases, GitLab Duo demonstrated high reliability, consistent prompt adherence, and practical outputs. Tasks that were bounded, declarative, and configuration-driven aligned well with the model's strengths. Unlike Java scenarios, no major execution failures or hallucinations were observed. These Python domains now serve as early success templates for safe and impactful AI adoption in enterprise workflows.

Table VI: Python Toolchain Uplift Observations

Tool	**Pre-Duo State**	**Post-Duo State**
Python Version	Mixed (3.7 in config, 3.10 in Docker/GitHub Actions)	Unified to 3.10 across all configs
Py.toml	Outdated `requires-python = >=3.7,<3.8`	Updated to reflect Python 3.10
MyPy / Ruff	`python_version = 3.7`, `target-version = py37`	Aligned to 3.10
Test Suite	Golden values outdated or incomplete	Refreshed values, improved assertions
Function Boundaries	Single-function monolith	6+ modular, typed, docstringed functions
CI/CD Outcome	Not evaluated	Passed all tests post-refactor

Use Case 6 - Generation of Code for a New Application

Beyond framework / SDK upgrades and extending legacy codebases, generation of new applications or modules represents a relatively significant proportion of developer time. This is the case where teams develop decoupled modules such as Multi Channel Applications (MCAs) and Interactive Voice Response (IVR) modules, that are built to specific patterns and integrate with back end services following familiar patterns (e.g. APIs confirming to Company API standards), or are developing prototypes. Automated generation of the code for such applications from a human readable requirement has the potential to save significant time above the use of automated bootstrapping tools, so long as the code generated sufficiently meets the requirement specified, is accurate to the required patterns and is of sufficient quality. The scope of the project limits us from testing out code generation scenarios on the Company network and to a Company defined application pattern. So this experiment is limited to a more greenfield scenario with a basic requirement.

The use case is focused on:

- Understanding of a human readable requirement for a basic application
- Accuracy of generated application to the requirement
- Quality of generated artefacts (structure, code, documentation, tests)
- Appropriate use of patterns and selection of dependencies
- Quality of build and run instructions
- How effectively issues encountered can be diagnosed and fixed

This is a very general greenfield scenario and more work would be needed against real Company application patterns for a more accurate assessment of suitability.

Use Case 6.1 – Build a new API Back End and Corresponding Front End for a Specific Requirement

The scenario was to create a simple API, back end and web app for a fictional development team productivity tracking app.

The 'back end' is just the code. No database was used – this was deliberate, to keep the scope simple. Instead, an 'in code' option using stubs and sample data was requested.

The back end was to be in Go and the front end in React. Go was chosen as an initial run to minimise the chances of issues with dependencies. The latest stable versions of Go and React were requested.

The initial prompt was fairly basic in what it requested. The requirement was for an 'app to track development team metrics', with the metrics to include team velocity and DORA metrics (without specifying any detail on which DORA metrics). The prompting used simple one-line statements for what was needed (in line with the examples of the Duo Workflow User Guide) such as:

> Include the following: 1. A data access layer for the data, using stubs in Go with sample data in code...

The Full Initial Prompt

> "Initialize a new Go project for an app to track development team metrics.
>
> Use the simple_crud_app_creation_go project that has been created, and the initial_attempt branch.
>
> **Metrics to track include:**
>
> - Team Velocity by month
> - DORA metrics by month
>
> **The project should include:**
>
> 1. A data access layer for the data, using stubs in Go with sample data in code
> 2. A REST API for accessing and updating the data
> 3. A web front end based on REACT and JavaScript for displaying the data
> 4. Some sample data to use when running the app
> 5. Some graphs in the web pages to show trends
>
> **Include the following:**
>
> 1. Update the README.md with project overview and setup instructions
> 2. Set up a basic CI/CD configuration
> 3. Add unit tests for the code
> 4. Use the latest stable versions of React and Go
> 5. Explain any dependencies included
> 6. A build guide clearly explaining dependencies, and specific instructions on how to build and run the application
>
> Minimise the dependencies and don't include any logging or logging framework.
>
> Explain the plan and document decisions and reasoning."

There was room for Duo Workflow to suggest the best architecture and dependencies to use, although it had been asked to "minimise the dependencies." The dependencies it selected to use were clearly stated.

The whole process of creating the app and getting it working took place over several workflows and interactions with Duo Workflow, and included:

- Creating a plan
- The initial build
- Refactoring the front end to remove any dependency on Node.js and npm
- Improving the ReadMe
- Fixing the 404 error when initially trying to access the app
- Manually fixing compilation issues in the Go code – they were limited in number and simple to spot, so Duo Workflow was not used to fix them

At each stage, GitLab Duo Workflow provided both summaries of what it had understood, what it proposed to do in summary form, and detailed plans with the specific changes listed.

Technical Decisions and Justifications

Backend (Go)
Architecture: I'll use a clean architecture approach with clear separation of concerns:

- Models: Define data structures
- Store: Data access layer with in-memory implementation (stubs with sample data)
- API: REST API handlers
- Middleware: Cross-cutting concerns

Dependencies:

- `gin-gonic/gin`: Popular, performant web framework for creating the REST API
- `go-playground/validator`: For input validation
- `rs/zerolog`: Fast, structured logging
- `golang.org/x/time/rate`: For rate limiting
- `stretchr/testify`: Testing utilities

Testing: Comprehensive unit tests for all components

Frontend (React)
Architecture: Component-based architecture using modern React practices:

- Functional components with hooks
- Context API for state management
- Reusable UI components

Dependencies:

- React 18.x (latest stable version)
- React Router for navigation
- Axios for API communication

- Chart.js for data visualization
- Material-UI or Tailwind CSS for UI components

Testing: Jest and React Testing Library for component tests

Snapshot of the generated project structure:

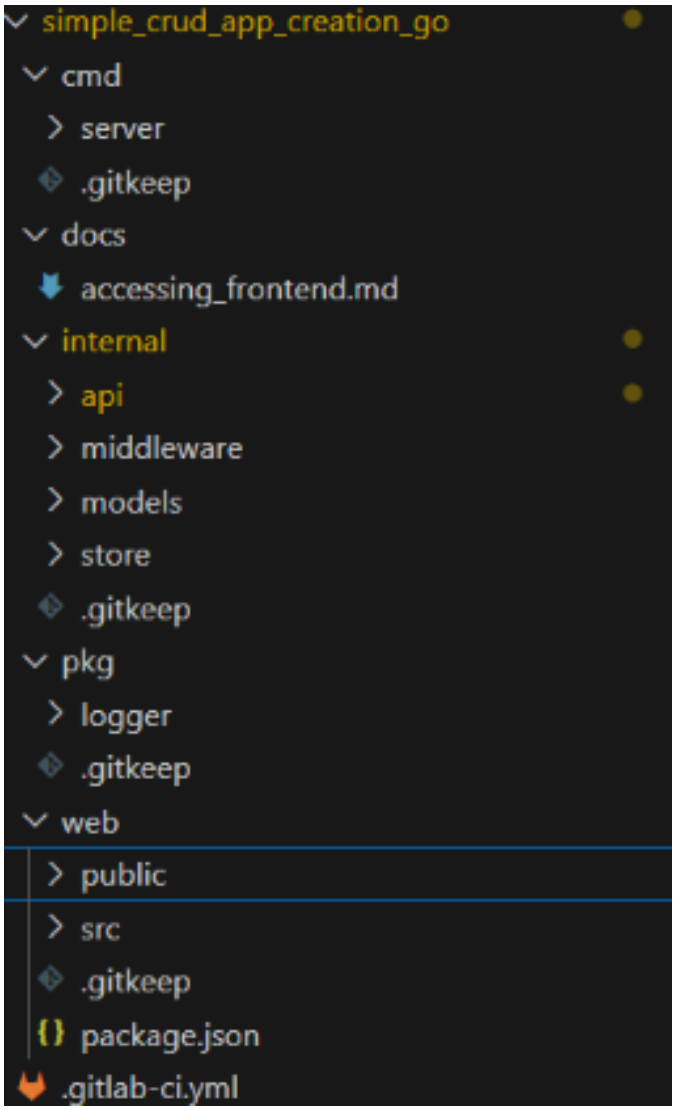

Figure V: Project Structure

Before executing the plan, GitLab Duo Workflow asked the following clarifying questions:

> I've analyzed your task description for clarity and completeness. While the core requirements are well-defined, there are a few areas that could benefit from additional clarification to ensure the task is implemented correctly.
> I'm ready to help with your project but I need a few key details:
>
> 1. Could you specify what specific DORA metrics should be tracked (e.g., deployment frequency, lead time for changes, change failure rate, mean time to restore)?
> 2. Are there any specific data visualization requirements for the graphs (e.g., line charts, bar charts, specific time ranges)?
> 3. What authentication/authorization requirements exist for the API, if any?
> 4. Are there any specific performance requirements or constraints for the application?
> 5. Should the application include any specific testing data scenarios or test coverage requirements?

All question responses were respected in the generated code.

The plan was executed as described, although:

- Node.js was not mentioned as a dependency for the front end.
- React 18 was used and mentioned as the latest stable version – it isn't.
- The statement on the back end code – ``Testing: Comprehensive unit tests for all components" – was very bold. As per later commentary, generated test coverage was poor.

We then progressed by asking GitLab Duo Workflow to refactor the front end to remove any dependency on Node.js – in order to keep it simple. After answering some questions, it proceeded to do this, but didn't update the build and run instructions. This was successfully completed after a following prompt.

GoLang Errors There were a small handful of compilation errors in the Go code. Some files had import declarations for packages that were not used, and a function that should have been declared public was private. These were easily diagnosed and corrected by hand. Following fixing these issues, the back end could be built and run as per instructions.

404 Error Accessing the Front End Upon accessing the URL for the front end as per run instructions, a HTTP 404 error was the result. GitLab Duo Workflow was asked to find the defect:

> After running the server I can't access the front end app at `[http://localhost:8080]` (http://localhost:8080) – instead I get a http 404 response. Find the defect

GitLab Duo Workflow asked some questions which were answered, and after its analysis it produced a good bug report that accurately identified the issue. An extract from the report is as follows:

> **Bug Report: Frontend 404 Error**
> **Issue Description**
> When running the Go server application, trying to access the frontend at `[http://localhost:8080]`(http://localhost:8080) results in a 404 error.
> **Root Cause**
> The router is configured to serve static files from the `./web/build` directory, but this directory doesn't exist in the repository. The frontend hasn't been built, so there are no compiled static files to serve.
> **Detailed Analysis**
>
> 1. In `internal/api/router.go`, the static file server is configured with:

```
fileServer := http.FileServer(
http.Dir("./web/build"))
r.Handle("/\*", http.StripPrefix("/",
fileServer))
```

2. The `web/build` directory doesn't exist in the repository, as confirmed by using the `list_dir` tool.

The root cause analysis was good in that it found the issues and identified the fix, but still referred to npm in the build steps which should have been removed.

Follow-up Prompt:

> The fix asks me to build the front end with npm. But npm should have been removed as a dependency. Remove the dependency on npm and provide a fix that doesn't depend on it.

This resulted in a plan which when executed removed npm as a dependency. The front end could then be successfully accessed.

Generated Code The server side code was decoupled into HTTP handler functions (in the `api` package), the domain model (`models` package), and the in-memory back end (`memory store` package), along with supporting packages for logging and launching the HTTP container.

Generated test coverage was minimal and patchy. Indeed, statement coverage was a mere 7.2 percent. For instance in the `api` package a single test file was generated seemingly to test the whole package (supporting comments clarifying this would have been helpful). But it missed some tests – the `TestTeamHandler` function missed tests for the `GetTeam`, `UpdateTeam` and `DeleteTeam` functions, and no tests were included at all for the `dora_handler.go` file. Many of the generated unit tests also failed first execution.

Several attempts were then made to get GitLab Duo Workflow to analyse the Go unit test coverage and improve it. Each attempt did a good job of analysing the gaps and produced good plans for improving, with clear success criteria. But frustratingly, each attempt to implement got stuck when executing the plan before any new tests were generated, and had to be abandoned. This includes when limiting scope to specific folders. Eventually some success was seen with targeting the prompt at a specific file. This succeeded in creating new tests which were clearly written, used good practices like table tests, and took the coverage from 0 percent for the file to over 90 percent. The one issue seen was that duplicate tests were created (with the same name).

Below is a sample of generated Go code, for one of the HTTP handler functions in `teams_handler.go`:

```go
// CreateTeam creates a new team
func (h *TeamHandler) CreateTeam(w http.ResponseWriter, r *http.Request) {
    var team models.Team

    if err := json.NewDecoder(r.Body).Decode(&team); err != nil {
        log.Error().Err(err).Msg("Failed to decode team")
        respondWithError(w, http.StatusBadRequest, "Invalid request payload")
        return
    }

    // Set timestamps
    now := time.Now()
    team.CreatedAt = now
    team.UpdatedAt = now

    // Set ID if not provided
    if team.ID == "" {
        team.ID = models.GenerateID()
    }

    if err := h.store.CreateTeam(team); err != nil {
        log.Error().Err(err).Interface("team", team).Msg("Failed to create tea
        respondWithError(w, http.StatusInternalServerError, "Failed to create
        return
    }

    respondWithJSON(w, http.StatusCreated, team)
}
```

FIGURE VI: GENERATED GO CODE

In general the generated code was clearly structured and the comments were good.

Summary Observations

The initial analysis completed in minutes and produced a clear plan that was true to the instruction. The plan included:

- The directory structure and files that would be created
- Summaries of technical decisions and justifications for the architecture, dependencies and testing, for the front and back ends
- An implementation plan with summaries of ordered tasks
- The data model that the app would be built to support
- The API endpoint details
- The CI/CD configuration

Where Duo Workflow needed more information, the questions were generally clear and the answers recognised.

After initial creation, the front end was successfully refactored to remove any dependency on Node.js, although the build instructions in the ReadMe were not initially updated to reflect this (they were successfully updated following another prompt).

The directory structure for the Go module was consistent with layout practices specific here – Organizing a Go module – The Go Programming Language.

The data model included detail that was fleshed out by GitLab Duo, e.g. for the Team Velocity metric it included detail such as planned story points, completed story points and average story points per team member.

A small number of compilation issues were seen in the Go code – these were import statements for packages that were not used in a file, and in one case a private function should have been public as it was called by other packages. These were easy and quick to fix by hand.

An issue in the generated web app was successfully analysed and fixed by Duo Workflow from a basic prompt asking for help analysing the defect (404 error when trying to access the app). There was a missing reference to the index.html file

which meant the app home page would not load. This is a relatively simple issue to diagnose but was done successfully with a clear bug report.

Overall structure and readability of the code was good, but test coverage poor.

Clarity of the generated ReadMe was good, although needed some additional prompts to improve instructions on running the app.

Go 1.22 was used by Duo Workflow, despite 1.24 being the latest stable version. React 18 was used, despite 19 being the latest.

The initial code generation completed in under half an hour.

Configuration artefacts and instructions for running in Docker were provided (though not requested) – these were not evaluated due to time constraints.

Conclusion for Use Case 6

Overall, the time saved versus coding the equivalent application by hand was significant, even for a small application like this.

Despite a small number of issues which were fixed through a combination of Duo Workflow and fixing by hand, the app compiled and ran successfully.

After the initial build, Duo Workflow successfully refactored the web app to remove any use of Node.js and npm, and fixed an issue in the web app.

Duo Workflow presented clear plans and gave confidence that it knew what it was doing and understood the brief.

More testing would be required on specific Company application patterns to see if they could be 'learned', and how it would cope with creating a larger scale application.

General Observations on the Tool / VS Plugin

Some complex workflows ̀hang' after a period of time, in that GitLab Duo Workflow appears busy but nothing is seemingly happening. In some cases the workflow has actually finished and the summary can be seen but only by refreshing the plugin or restarting VSCode.

Workflows can refer to previous workflows by ID. On some occasions it will not recognise a previous workflow, despite the ID being correct.

Workflows are not atomic — if you stop a workflow part way through executing a plan, changes will not be rolled back.

Pressing return in the prompt text box kicks off the workflow. It doesn't do carriage return.

Sometimes a workflow will lose context of the branch it is working in. On one occasion it successfully performed analysis on existing code but lost the context of the local branch when attempting to execute the plan after it had been approved.

Threats To Validity

In this section, we discuss the threats to the validity of our work and case studies.

Our case study does not involve a controlled experiment comparing the LLM-based approach to other techniques, making it difficult to isolate the specific contribution of the different components of the system to the observed improvements.

Additionally, our case study was conducted within the specific context of Java, Python and Node, which may limit the generalizability of the findings to other teams. Our case study was conducted by 5 developers, which may not be representative of the full range of other types of code migration tasks and developer experiences.

The study was based on the GitLab Workflow Duo using the Anthropic Claude version 3.5. The generalizability of our results on migrations to other LLMs may be limited.

Key Findings

Java and Dependent Library Upgrade

Java-based use cases presented a sharper contrast. Despite the relatively simple and clean application, Gitlab Duo Workflow encountered challenges when faced with this layered architecture, build tool complexity, and implicit configurations typical of Spring Boot and legacy Java applications. Prompt interpretation required high specificity, and agent behavior was inconsistent—ranging from partial completions to hallucinated references. However, Duo did show promise in smaller-scale tasks (JavaDoc, etc), where the scope was tightly constrained.

However, Gitlab Duo Workflow demonstrated a good summary of the structure of the code base and the nature of the technologies used, which is the foundational step for creating a good plan on how to migrate. The plan itself is the second point that Gitlab Duo Workflow appears to be relatively sensible and requires a moderate level of rectification that seems to be inversely correlated to the level of details provided in the prompt.

The aspects that influenced our scoring downwards in those early days of private beta are all related to various aspects of performance, where it is stability, speed of ingesting file content as well as accuracy of the outcome and its current additional value in comparison to just GitLab Duo Chat.

Score: 2.0 / 5

Unit Test Generation (Java)

The focus for this use case was to understand how Gitlab Duo Workflow would perform against legacy code. Overall, the generated tests reflected what would be expected of seasoned Java developers. While there were small points to quibble over they are largely negligible within the context of having to change legacy code and using characterisation tests as a means of doing so safely.

However, when the overall goal is to uplift significant portions of an application then the longer feedback loop becomes concerning. Added to the fact that the PetClinic codebase is of drastically lower complexity than what is found in The Company, then a more interactive approach such that progress can be inspected by the human in the loop as code changes towards the overall goal are being made would be preferable.

Score: 3.0 / 5

FastAPI Modernization (Code Quality, Test Generation, CI Readiness)

This use case explored Gitlab Duo Workflows' ability to uplift an existing FastAPI codebase through modularization, documentation, and test generation. The AI succeeded in restructuring the code into clearly separated functions, applying consistent type hints, and generating useful NumPy-style docstrings. It also helped regenerate test cases and validate the repo's CI/CD compliance in a clean pass, demonstrating functional correctness.

However, the strength of the output was closely tied to the prompt's specificity. The scope was relatively narrow, and success relied on the code being already self-contained and idiomatic. While the results were reliable, this success may not generalize to more abstract or loosely structured Python services.

Score: 3.5 / 5

FastAPI Migration Planning (Python 3.7 → 3.10 + Dependency Alignment)

This scenario tested Gitlab Duo Workflows' analytical capability rather than direct code transformation. The AI agent reviewed the configuration environment—including pyproject.toml, Dockerfiles, and static analyzers—and correctly identified mismatches in Python versions and dependency constraints. It produced a structured migration plan with supporting artefacts, including a compatibility matrix, risk assessment, and update checklist. The outputs reflected a solid understanding of ecosystem-level changes, such as Pydantic v2's impact on SQLModel.

The execution, however, remained advisory. Gitlab Duo Workflow did not directly apply changes or validate runtime compatibility. Still, the planning-level output was clear, actionable, and aligned with best practices—making it a valuable tool for teams in the early stages of modernization planning.

Score: 4.0 / 5

Python Jupyter: Financial Model Refactoring and CI/CD Enablement

This use case evaluated GitLab Duo Workflow's ability to autonomously transform standalone, exploratory Python scripts—typically authored in Jupyter—into modular, CI/CD-compliant codebases. Starting from a basic algorithmic script, Duo generated all required artifacts, including tests and configuration files, to successfully pass a GitLab CI pipeline on the first attempt.

The test then progressed to a more complex financial model (binomial option pricing), where GitLab Duo Workflow again delivered structured refactoring, type annotations, NumPy-style docstrings, and preserved functional behavior. It regenerated tests, aligned validation logic, and ensured CI integrity across transformations—without hallucinated dependencies or broken compatibility.

GitLab Duo Workflow's prompt interpretations were consistent, and it handled edge cases like floating-point variations through test updates. The system maintained API contracts and required minimal intervention throughout.

This scenario highlights GitLab Duo Workflow's maturity in managing the full DevOps cycle—from refactor to validation—for scientific Python workloads.

Score: 4.0 / 5

Greenfield Application Generation (Go + React)

This scenario tested Duo Workflow's ability to autonomously generate a new full-stack application from a loosely defined human-readable prompt. The objective was to scaffold a productivity tracking app with a Go backend and a React frontend, including CI/CD setup, REST API endpoints, test coverage, and frontend chart visualizations. Duo responded with structured plans and complete project artifacts, including a working folder structure, a layered architecture, React components, sample data, and build/run instructions.

It correctly adopted modular practices in both frontend and backend architecture. The tool also handled followup requests, such as removing Node.js/npm dependencies, with targeted execution and issue resolution. Some limitations were observed in test coverage, with generated tests lacking coverage depth and failing on first execution. Attempts to auto-fill coverage gaps via prompts often stalled. Minor inconsistencies in dependency versions (e.g., outdated React and Go releases) also emerged.

Despite these issues, the scenario showcased Duo's ability to generate reasonably complete and production-adjacent application scaffolds with minimal intervention. The agent provided coherent architectural decisions and maintained continuity across prompts, indicating maturity in greenfield synthesis workflows. Demonstrates clear potential for use in rapid prototyping and boilerplate generation.

Score: 3.0 / 5

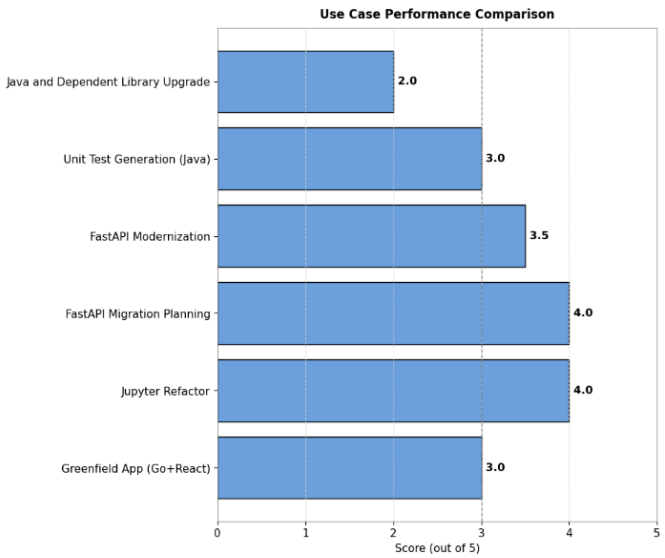

Figure VII: Performance Comparison

Figure VII above, reflects the performances of the use cases with the respective scores

Recommendations and Next Steps

Our recommendation for next steps are:

- Continue experimentation, working closely with GitLab on upcoming improvements.
- Run structured comparisons with other similar code gen-

eration tools, e.g., Windsurf, Junie, etc.

- Run structured comparisons of the same use cases on different technology stacks.
- Explore further use cases.
- Run comparisons against real Barclays code and application patterns.

From there, opportunities for active trials on real but low-risk projects will start to emerge. We urge caution with this—more small steps are required before getting to wider roll-out. GitLab Duo Workflow and its competitors are evolving very quickly. We saw changes in the tool during our short trial. Therefore, we are likely to see improvements, potentially over a short span of time, to the use cases we ran and similar scenarios.

We urge staying close to this progress through active working groups.

Acknowledgement

We would like to thank our sponsors for their continued support of this project work. We also extend our sincere gratitude to GitLab for providing early access to the Duo Workflow beta environment, which enabled the authors to perform hands-on evaluation of its capabilities.

References

[1] Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., ... & Zhang, Y. (2023). Sparks of Artificial General Intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*. https://doi.org/10.48550/arXiv.2303.12712

[2] Pearce, H., Ahmad, F., Ghaffari, P., & Ernst, M. D. (2023). Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *IEEE Symposium on Security and Privacy*, 1–18. https://doi.org/10.1109/SP46214.2023.00033

[3] Sarkar, A., & Spinellis, D. (2022). A Catalog of Prompting Techniques to Improve Code Generation with Large Language Models. *arXiv preprint arXiv:2212.09561*. https://doi.org/10.48550/arXiv.2212.09561

[4] Vaithilingam, P., Ahmad, W. U., Svyatkovskiy, A., Sundaresan, N., & Chattopadhyay, S. (2022). Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by LLMs. *ESEC/FSE '22*, 286–297. https://doi.org/10.1145/3540250.3549133

[5] Sobania, D., Liu, S., Nguyen, A., & Chattopadhyay, S. (2023). An Empirical Study of Software Developers' Prompt Engineering Practices for Code Generation. *CHI '23*, 1–15. https://doi.org/10.1145/3544548.3581049

[6] Bird, C., Zimmermann, T., & Nagappan, N. (2015). The Art of CI/CD: Modeling Software Delivery Performance in Large Organizations. *IEEE Software, 32*(2), 45–52. https://doi.org/10.1109/MS.2015.38